Guía paso a paso: Flujo completo con Git y Azure DevOps

Voy a darte el flujo completo, desde cero, con cada comando explicado. Asumamos que ya tienes el repositorio clonado y estás en la carpeta del proyecto en Git Bash.

📌 Paso 1: Verificar en qué rama estás

Abre **Git Bash** y navega a la carpeta de tu proyecto:

```bash
cd /c/Users/TuUsuario/Documents/Proyectos/tu-repositorio
```

Verifica la rama actual:

```bash
git branch
```

Lo que verás:

```text
* main
  remotes/origin/main
```

El asterisco `*` indica que estás en `main`. Ese es el punto de partida.

🌿 Paso 2: Crear tu nueva rama

```bash
git checkout -b feature/mi-rama-nueva
```

¿Qué hace? Crea una rama nueva llamada `feature/mi-rama-nueva` y te cambia a ella automáticamente. **No copia archivos, no descarga nada**, simplemente mueve el "puntero" de Git a una nueva rama que apunta al mismo contenido que `main`.

Verifica que estás en la rama nueva:

```bash
git branch
```

Lo que verás:

```text
  main
* feature/mi-rama-nueva
```

El asterisco ahora está en tu rama nueva.

Verifica que los archivos están ahí:

```bash
ls -la
```

Vas a ver todos los archivos del proyecto (los mismos que había en `main`). Eso confirma que **tu rama nueva ya tiene todos los archivos de main**.

📤 Paso 3: Subir la rama nueva al servidor (sin cambios aún)

Aunque no hayas tocado ningún archivo, debes "publicar" la rama en Azure DevOps para que exista allá:

```bash
git push -u origin feature/mi-rama-nueva
```

¿Qué hace este comando?

•`git push`: Envía al servidor.
•`-u origin`: Le dice a Git que la rama nueva en el servidor (origin) será la "oficial" para esta rama local. Esto te permite escribir solo `git push` a partir de ahora.
•`feature/mi-rama-nueva`: El nombre de tu rama.

Lo que verás:

```
text
```

```
* [new branch]      feature/mi-rama-nueva -> feature/mi-rama-nueva
Branch 'feature/mi-rama-nueva' set up to track remote branch 'feature/mi-rama-nueva' from
'origin'.
```

Verifica en Azure DevOps:

1.Ve a la web de tu repositorio.
2.Menú izquierdo → **Branches**.
3.Debes ver tu rama `feature/mi-rama-nueva` junto a `main`.

✏ Paso 4: Hacer cambios en los archivos

Ahora sí, abre tus archivos con tu editor (PL/SQL Developer, Forms Builder, VS Code, Notepad++) y modifica lo que necesites. Por ejemplo:

1.Abre `forms/FOR_BLOQUEO_PV.fmb` y modifica un trigger.
2.Abre `scripts/mi_paquete.sql` y agrega un procedimiento nuevo.
3.Guarda los cambios.

🔍 Paso 5: Verificar qué cambió

Vuelve a Git Bash y ejecuta:

```
bash
```

```
git status
```

Lo que verás (ejemplo):

```
text
```

```
On branch feature/mi-rama-nueva
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   forms/FOR_BLOQUEO_PV.fmb
```

```
        modified:   scripts/mi_paquete.sql

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        nuevo_archivo.sql
```

Significado:

•**Modified (en rojo):** Archivos que ya existían y modificaste.
•**Untracked (en rojo):** Archivos nuevos que Git todavía no conoce.

Si `git status` te dice "nothing to commit, working tree clean", significa que **no has modificado nada**. En ese caso, no hay cambios que subir.

📦 Paso 6: Preparar los cambios (staging)

Ahora dile a Git qué archivos quieres incluir en el "paquete":

Opción A — Todos los cambios:

```bash
git add .
```

Opción B — Solo archivos específicos:

```bash
git add forms/FOR_BLOQUEO_PV.fmb
git add scripts/mi_paquete.sql
git add nuevo_archivo.sql
```

Verifica que se prepararon:

```bash
git status
```

Lo que verás ahora:

```text
```

```
On branch feature/mi-rama-nueva
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   forms/FOR_BLOQUEO_PV.fmb
        modified:   scripts/mi_paquete.sql
        new file:   nuevo_archivo.sql
```

Los archivos ahora aparecen en **verde** (staged). Están listos para el commit.

💾 Paso 7: Crear el commit

```bash
git commit -m "feat: agregar bloqueo automatico de puntos de venta"
```

¿Qué hace? Agrupa todos los cambios preparados en un "paquete" con un mensaje descriptivo. Este paquete se guarda **localmente** (todavía no está en Azure).

Lo que verás:

```text
[feature/mi-rama-nueva abc1234] feat: agregar bloqueo automatico de puntos de venta
 3 files changed, 45 insertions(+), 12 deletions(-)
```

Buenas prácticas para el mensaje:

Prefijo	Cuándo usarlo
`feat:`	Nueva funcionalidad
`fix:`	Corrección de bug
`docs:`	Documentación
`refactor:`	Reorganización sin cambio funcional
`chore:`	Tareas varias

🚀 Paso 8: Subir el commit a Azure DevOps

```bash
git push
```

¿Qué hace? Envía tu commit al servidor. Como ya configuraste `-u` en el paso 3, no necesitas especificar la rama.

Lo que verás:

```text
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Writing objects: 100% (5/5), 1.2 KiB | 1.2 MiB/s, done.
To https://dev.azure.com/...
   abc1234..def5678  feature/mi-rama-nueva -> feature/mi-rama-nueva
```

Verifica en Azure DevOps:

1.Ve a la web → **Branches**.
2.Haz clic en tu rama `feature/mi-rama-nueva`.
3.Verás los archivos y commits actualizados.

🔄 Paso 9: Traer los últimos cambios de `main` a tu rama (merge)

Imagina que otra persona trabajó en `main` y subió cambios. Tú quieres traer esos cambios a tu rama para no quedarte atrás.

9.1 — Verifica si `main` tiene cambios nuevos

```bash
git fetch origin
git log HEAD..origin/main --oneline
```

Si aparece una lista de commits, significa que `main` tiene cambios que tú no tienes. Si no aparece nada, `main` no ha cambiado.

9.2 — Traer los cambios de `main` a tu rama

```bash
git pull origin main
```

¿Qué hace? Descarga los cambios de `main` y los fusiona (merge) con tu rama.

9.3 — Si hay conflictos

Si Git te dice:

```text
CONFLICT (content): Merge conflict in archivo.sql
Automatic merge failed; fix conflicts and then commit the result.
```

Significa que **tú y otra persona modificaron las mismas líneas** del mismo archivo. Tienes que resolver el conflicto manualmente:

a) Abre el archivo conflictivo con tu editor. Verás marcas así:

```sql
<<<<<<< HEAD
-- TU VERSIÓN (rama feature/mi-rama-nueva)
SELECT * FROM punto_venta WHERE estado = 'ACTIVO';
=======
-- VERSIÓN DEL SERVIDOR (rama main)
SELECT * FROM punto_venta WHERE estado = 'INACTIVO';
>>>>>>> origin/main
```

b) Decide qué conservar:

- **Conservar tu versión:** Borra el bloque `=======` a `>>>>>>> origin/main`.
- **Conservar la versión de main:** Borra el bloque `<<<<<<< HEAD` a `=======`.
- **Combinar ambas:** Escribe una nueva línea que combine lo mejor de las dos.

c) Borra también las líneas `<<<<<<<`, `=======`, `>>>>>>>`.

d) Marca el conflicto como resuelto:

bash

```
git add archivo.sql
```

e) Termina el merge con un commit:

bash

```
git commit -m "merge: resolver conflictos con main"
```

f) Sube los cambios:

bash

```
git push
```

📊 Paso 10: Flujo diario completo (resumen)

Cada vez que vayas a trabajar en tu rama, sigue este orden:

bash

```
# 1. Ir a la carpeta del proyecto
cd /c/Users/TuUsuario/Documents/Proyectos/tu-repositorio

# 2. Asegurarte de estar en tu rama
git checkout feature/mi-rama-nueva

# 3. Traer lo último de tu rama
git pull origin feature/mi-rama-nueva

# 4. Traer lo último de main (opcional, pero recomendado)
git pull origin main

# 5. Editar archivos con tu editor

# 6. Ver qué cambió
git status

# 7. Agregar los cambios
git add .
```

```
# 8. Hacer el commit
git commit -m "feat: descripción del cambio"

# 9. Subir
git push
```

🎯 Verificación final

Al terminar, verifica en Azure DevOps:

1. **Branches:** Tu rama `feature/mi-rama-nueva` está publicada.
2. **Commits:** Haz clic en tu rama y verás tu commit con el mensaje que escribiste.
3. **Files:** Los archivos modificados están actualizados.
4. **Pull Request (opcional):** Si quieres fusionar tu rama con `main`, ve a **Pull Requests → New Pull Request**, selecciona `feature/mi-rama-nueva` como origen y `main` como destino, y solicita la revisión.

⚠ Errores comunes y cómo resolverlos

Error	Causa	Solución
`nothing to commit, working tree clean`	No has modificado nada	Abre un archivo, modifícalo y guarda
`Your branch is behind 'origin/main'`	Tu rama está atrasada	`git pull origin main`
`CONFLICT` al hacer pull	Alguien modificó las mismas líneas	Resolver manualmente (paso 9.3)
`Permission denied`	Token expirado	Crear nuevo PAT en Azure DevOps
`fatal: not a git repository`	No estás en la carpeta del proyecto	`cd` a la carpeta correct

Ese programa que tienes abierto es **Git GUI**, la interfaz gráfica oficial de Git (la que viene incluida). Es muy útil porque te ahorra escribir comandos en la consola.

Para hacer exactamente lo mismo que en los comandos de consola, pero con esta interfaz, sigue estos pasos:

🌿 Paso 1: Crear tu nueva rama

Como te dice en la esquina superior izquierda, actualmente estás en `main`. Vamos a crear la nueva rama:

1.En el menú superior, haz clic en `Branch` (Rama).
2.Selecciona `Create...` (Crear...).
3.En la ventana que aparece, escribe el nombre de tu rama (por ejemplo: `feature/mi-rama-nueva`).
4.Asegúrate de que la opción **"Checkout After Creation"** (Cambiar a ella después de crearla) esté marcada.
5.Haz clic en `Create`.
(Verás que arriba a la izquierda ahora dice `Current Branch: feature/mi-rama-nueva`).

✏ Paso 2: Editar los archivos

Igual que antes. Abre tus archivos (`.sql`, `.fmb`, etc.), haz tus cambios, guárdalos y cierra los editores.

🔄 Paso 3: Actualizar y Preparar (Stage) los cambios

1.Vuelve a la ventana de **Git GUI**.
2.Haz clic en el botón `Rescan` (abajo a la izquierda). Esto obligará a Git a buscar los archivos que modificaste.
3.En el cuadro de arriba ("Unstaged Changes" / Cambios no preparados) aparecerán tus archivos en rojo.
4.Haz clic en el botón `Stage Changed` (Preparar cambios).
(Los archivos se moverán al cuadro de abajo, "Staged Changes (Will Commit)", en color verde).

💾 Paso 4: Hacer el Commit

1.En el cuadro de texto grande de la parte inferior llamado `Commit Message`, escribe el mensaje de tu commit (ej. `feat: agregar nueva pantalla de bloqueos`).
2.Haz clic en el botón `Commit`.
(Si te sale una ventana avisando que debes configurar tu nombre y correo, sigue las instrucciones que te da la misma ventana, escribe tu nombre y correo, y vuelve a darle Commit).

🚀 Paso 5: Subir los cambios a Azure (Push)

1.Haz clic en el botón `Push` (Empujar).
2.Se abrirá una ventana. En la parte superior, asegúrate de que el **Remote** sea `origin`.
3.**MUY IMPORTANTE:** En la sección **Destination** (Destino), verifica que esté seleccionada tu nueva rama (`feature/mi-rama-nueva`) y **NO** `main`.
4.Haz clic en el botón `Push` de esa ventana.

⚠ Si por alguna razón te pide usuario y contraseña

Azure te pedirá credenciales. Recuerda que:

•**Usuario:** Tu correo de la empresa o de Azure.
•**Contraseña:** El **Token (PAT)** que generaste en Azure DevOps (Azure ya no acepta la contraseña normal de tu cuenta de Microsoft).

¡Listo! Con estos 5 pasos gráficos haces exactamente el mismo flujo que con los comandos en negro, pero usando los botones de Git GUI.

Sí, **se puede y es lo más recomendado**. De hecho, en el mundo profesional se acostumbra a agrupar los scripts por tipo de objeto para facilitar la instalación.

No necesitas crear un archivo por cada tabla. Puedes tener un solo archivo llamado `01_crear_tablas.sql` con las 5 tablas, y otro llamado `02_crear_vistas.sql` con las 2 vistas.

Aquí te explico cómo hacerlo para que funcione a la primera y no te dé errores.

⚠ Reglas de Oro para scripts con múltiples objetos

1.**El orden importa:** Siempre crea primero las tablas y luego las vistas (las vistas dependen de las tablas).
2.**El punto y coma (`;`):** Cada sentencia `CREATE` debe terminar con un punto y coma.
3.**La barra (`/`):** Aunque para puros `CREATE TABLE` no es estrictamente obligatoria en algunos editores, es una **excelente práctica** poner una barra `/` al final de cada bloque para forzar su ejecución.
4.**Cómo ejecutarlo:** En PL/SQL Developer o SQL Developer, **NO** uses `F8` (ejecuta solo una línea). Debes seleccionar todo el texto (`Ctrl+A`) y usar `F5` (Run Script) o abrir una **Command Window**.

📄 Script 1: `01_crear_tablas.sql`

Aquí tienes el ejemplo con tus 5 tablas jerárquicas. Fíjate cómo las tablas "padre" se crean primero para que las llaves foráneas (FK) no fallen.

```sql
-- ============================================================================
-- SCRIPT 1: CREACIÓN DE LAS 5 TABLAS
-- Ejecutar primero este script.
-- ============================================================================

-- 1. Tabla CANAL
CREATE TABLE canal (
    id_canal NUMBER PRIMARY KEY,
    nombre_canal VARCHAR2(100) NOT NULL
);
/

-- 2. Tabla DIVISION (Depende de CANAL)
CREATE TABLE division (
    id_division NUMBER PRIMARY KEY,
    id_canal NUMBER NOT NULL,
```

```
    nombre_division VARCHAR2(100) NOT NULL,
    CONSTRAINT fk_div_canal FOREIGN KEY (id_canal) REFERENCES canal(id_canal)
);
/

-- 3. Tabla TERRITORIO (Depende de DIVISION)
CREATE TABLE territorio (
    id_territorio NUMBER PRIMARY KEY,
    id_division NUMBER NOT NULL,
    nombre_territorio VARCHAR2(100) NOT NULL,
    CONSTRAINT fk_terr_div FOREIGN KEY (id_division) REFERENCES division(id_division)
);
/

-- 4. Tabla AGENCIA (Depende de TERRITORIO)
CREATE TABLE agencia (
    id_agencia NUMBER PRIMARY KEY,
    id_territorio NUMBER NOT NULL,
    nombre_agencia VARCHAR2(100) NOT NULL,
    CONSTRAINT fk_agencia_terr FOREIGN KEY (id_territorio) REFERENCES
territorio(id_territorio)
);
/

-- 5. Tabla PUNTO_VENTA (Depende de AGENCIA)
CREATE TABLE punto_venta (
    id_punto_venta NUMBER PRIMARY KEY,
    id_agencia NUMBER NOT NULL,
    nombre_pv VARCHAR2(150) NOT NULL,
    estado VARCHAR2(10) DEFAULT 'ACTIVO',
    CONSTRAINT fk_pv_agencia FOREIGN KEY (id_agencia) REFERENCES agencia(id_agencia)
);
/

-- Confirmar los cambios
COMMIT;
```

Script 2: `02_crear_vistas.sql`

Una vez creadas las tablas, ejecutas este segundo script. Aquí puedes ver cómo las vistas consultan las tablas que acabas de crear.

sql

```sql
-- ============================================================================
-- SCRIPT 2: CREACIÓN DE LAS 2 VISTAS
-- Ejecutar DESPUÉS del script de tablas.
-- ============================================================================

-- 1. Vista completa de la jerarquía de Puntos de Venta
CREATE OR REPLACE VIEW v_pv_jerarquia AS
SELECT pv.id_punto_venta,
       pv.nombre_pv,
       pv.estado,
       a.nombre_agencia,
       t.nombre_territorio,
       d.nombre_division,
       c.nombre_canal
  FROM punto_venta pv
  JOIN agencia a    ON a.id_agencia = pv.id_agencia
  JOIN territorio t ON t.id_territorio = a.id_territorio
  JOIN division d   ON d.id_division = t.id_division
  JOIN canal c      ON c.id_canal = d.id_canal;
/

-- 2. Vista resumida para contar cuántos puntos de venta hay por canal
CREATE OR REPLACE VIEW v_resumen_pv_canal AS
SELECT c.nombre_canal,
       COUNT(pv.id_punto_venta) AS total_puntos_venta
  FROM canal c
  JOIN division d   ON d.id_canal = c.id_canal
  JOIN territorio t ON t.id_division = d.id_division
  JOIN agencia a    ON a.id_territorio = t.id_territorio
  JOIN punto_venta pv ON pv.id_agencia = a.id_agencia
 GROUP BY c.nombre_canal;
/

-- Confirmar los cambios
```

```
COMMIT;
```

💡 Un truco extra: ¿Qué pasa si lo ejecuto dos veces?

Si intentas ejecutar el `01_crear_tablas.sql` dos veces, Oracle te dará un error `ORA-00955: name is already used by an existing object` porque las tablas ya existen.

Para evitar eso en los pases a servidores (especialmente en desarrollo), a veces se pone un bloque al principio de todo para borrar las tablas si existen (con cuidado de borrarlas en orden inverso para no violar las llaves foráneas):

```sql
-- Pon esto al INICIO de tu script 01_crear_tablas.sql si quieres poder reiniciarlo sin errores
BEGIN
   EXECUTE IMMEDIATE 'DROP TABLE punto_venta CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN
   EXECUTE IMMEDIATE 'DROP TABLE agencia CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN
   EXECUTE IMMEDIATE 'DROP TABLE territorio CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN
   EXECUTE IMMEDIATE 'DROP TABLE division CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN
   EXECUTE IMMEDIATE 'DROP TABLE canal CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
```

(El `EXCEPTION WHEN OTHERS THEN NULL` evita que el script se detenga si la tabla no existe la primera vez).

📌 Resumen

Sí, crea los archivos `01_tablas.sql` y `02_vistas.sql`. Pega todo el código, guarda, y cuando vayas a instalarlo en otro servidor, abre una ventana de comandos en tu herramienta, ejecuta todo el texto de corrido, y verás cómo los 5 objetos y las 2 vistas se crean uno tras otro sin problemas.